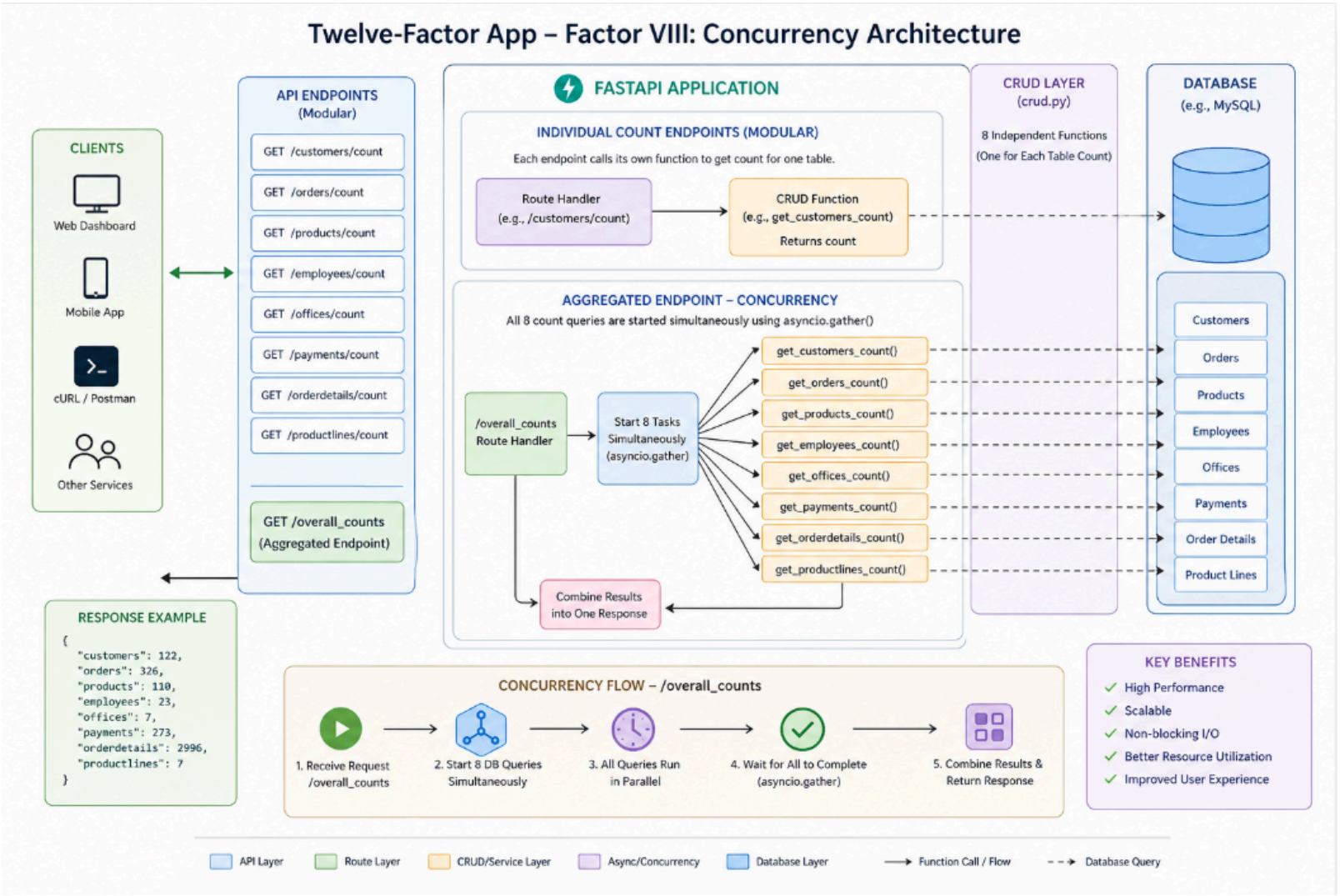
Mastering Factor VIII: Concurrency (Twelve-Factor App)



Objective

Build a high-performance API dashboard that retrieves record counts from multiple database tables **simultaneously** using concurrency. The goal is to avoid sequential delays and improve response speed.

Part 1: Create 8 Individual Endpoints (Modular Design)

Before building the dashboard, implement **eight separate API endpoints**. Each endpoint should return the **total number of rows** in a specific database table.

Endpoint Specifications

Table Name	Endpoint Path	Description
Customers	GET /customers/count	Returns total customers
Orders	GET /orders/count	Returns total orders
Products	GET /products/count	Returns total products
Employees	GET /employees/count	Returns total employees
Offices	GET /offices/count	Returns total offices
Payments	GET /payments/count	Returns total payments
Order Details	GET /orderdetails/count	Returns total order details
Product Lines	GET /productlines/count	Returns total product lines

Why this matters

This follows the principle of **Modularity**:

- Each endpoint is independent and reusable
- Users can request only what they need
- Improves maintainability and clarity

Part 2: Build the Aggregated Endpoint

Endpoint

GET /overall_counts

This endpoint acts as a **master dashboard**, returning counts from all tables in a single response.

Concurrency Requirement (VERY IMPORTANT)

You must NOT fetch data sequentially.

Wrong Approach (Sequential)

- Query Customers → wait
- Then Orders → wait
- Then Products → wait
  This is slow and inefficient.

Correct Approach (Concurrent)

Use Python's asyncio.gather() to run all database queries **at the same time**.

Key Requirements:

- Start all 8 queries simultaneously
- Use await asyncio.gather(...) to wait for all results
- Combine results into one response object

Expected JSON Response


```
{
 "customers": 122,
 "orders": 326,
 "products": 110,
 "employees": 23,
 "offices": 7,
 "payments": 273,
 "orderdetails": 2996,
 "productlines": 7
}
```

Logging Requirement (IMPORTANT)

Logging must be implemented across both individual and aggregated endpoints to ensure observability and debugging support.

Where Logging Should Be Used

- **Endpoint level (router.py)**
  - Log incoming requests for each /count endpoint
  - Log when /overall_counts is called
  - Log response status (success/failure)

- **CRUD layer (crud.py)**
  - Log each database count query
  - Log query execution start and completion
  - Log any database errors
- **Concurrency endpoint (/overall_counts)**
  - Log when all tasks are started
  - Log when asyncio.gather() completes
  - Log total response time for performance tracking

Conceptual Lesson: Factor VIII – Concurrency

The Twelve-Factor App emphasizes **scalability through concurrency**.

Sequential Processing

- One task at a time
- Like a single cashier serving a long queue
- Slow and inefficient under load

Concurrent Processing

- Multiple tasks run simultaneously
- Like multiple cashiers serving customers at once
- Much faster and scalable

Key Insight

Using async and await in FastAPI ensures:

> The server does not wait idle while the database responds—it continues processing other tasks.

Success Checklist

Modularity

- 8 separate functions exist in crud.py
- Each table has its own count function

Individual Endpoints

- Each /table/count endpoint works independently

Concurrency

- /overall_counts uses asyncio.gather()
- No sequential database calls are used

Robustness

- API handles empty tables gracefully
- Returns 0 instead of errors when no data exists

Logging

- All endpoints log requests and responses
- Database queries are tracked
- Concurrency execution time is recorded
